

Introduction to Spring & Spring Boot

"Build Better Backends"

1 The Starting Point : How Web Applications Communicate

Before understanding Spring, Spring Boot, Servlet or any backend framework, we first need to understand one basic question :

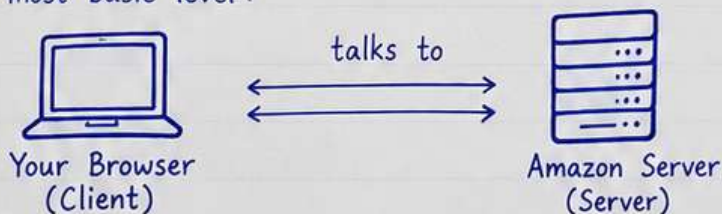
How does a user sitting on one machine communicate with code running on another machine ?

This is the foundation of web development.
When you open a browser and type :

`www.amazon.com`

your browser is running on your laptop or mobile phone, while Amazon's application is running on Amazon's server somewhere else.

At the most basic level :



This structure is called
Client-Server Architecture.

2 Client - Server Architecture

What is a Client ?

- A client is the side that asks for something.
- Examples of clients :
 - Browser
 - Mobile app
 - Postman
 - Frontend React app
 - Android app
 - iOS app

When you open a website, your browser becomes the client. It sends a request like :

"Hey server, give me this webpage."

What is a Server ?

- A server is the side that receives requests, processes them, and sends back a response.
- Examples of servers :
 - Amazon server
 - YouTube server
 - Bank server
 - Your Spring Boot application
- A server usually performs tasks like :
 - Checking login details
 - Fetching data from a database
 - Applying business rules
 - Saving information
 - Returning a response

The basic idea is simple :
Client asks. Server responds.

3 HTTP : The Language of the Web

Now the important question is :

- How does the browser know how to ask ?
- How does the server understand what the browser is asking ?

For communication to happen properly, both sides need a common language. That common language is called **HTTP**.

What is HTTP ?

- HTTP stands for :
HyperText Transfer Protocol

In simple words :

HTTP is the rulebook for communication between a client and a server.

HTTP defines :

- How a request should look
- How a response should look
- Which method is being used
- Which URL is being called
- What data is being sent
- Which status code is returned

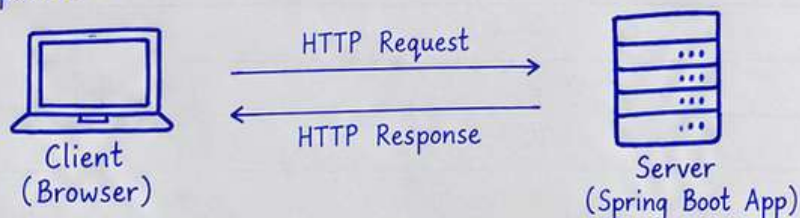
So the browser and server do not randomly exchange text. They follow a proper format.

HTTP is an application-layer protocol that works on top of TCP/IP.

4 Request - Response Cycle

Every web interaction follows the same basic pattern:

- 1 Client sends request
- 2 Server processes request
- 3 Server sends response
- 4 Client displays or uses the response



Client asks. Server responds.

Example :

When you open :

`www.coderarmy.in/courses`

The browser may send an HTTP request like :

```
GET /courses HTTP/1.1
Host: www.coderarmy.in
```

- The server receives the request and understands :
- The user wants the courses page.
 - Fetch the courses data.
 - Prepare the response.
 - Send it back to the browser.

The server may send back :

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>
  ... course data ...
</html>
```

The browser receives the response and displays the page.

5 Anatomy of an HTTP Request

An HTTP request usually contains four main parts:

- 1 Method
- 2 URL or path
- 3 Headers
- 4 Body (optional)

Example :

```
POST /login HTTP/1.1
Host: www.coderarmy.in
Content-Type: application/json

{
  "email": "abc@gmail.com",
  "password": "12345"
}
```

Meaning :

- POST** → I am sending data
- /login** → I want to call the login functionality
- Headers** → Extra information about the request
- Body** → Actual data being sent

6 HTTP Methods

The first line of an HTTP request is called the **request line**.

It contains the method, path, and HTTP version.

The method tells the server what action the client wants to perform.

Method	Meaning	Example Use
GET	Read data	Fetch a list of orders
POST	Create data	Place a new order
PUT	Replace data completely	Update an entire user profile
PATCH	Update data partially	Change only the phone number
DELETE	Remove data	Cancel an order

When to use what ?

- Use **GET** when you only fetch data.
- Use **POST** when you create something.
- Use **PUT** when you replace the whole resource.
- Use **PATCH** when you update a part of the resource.
- Use **DELETE** when you remove data.

7 HTTP Headers

Headers are key-value pairs that provide extra information about the request.

They tell the server things like :

- What format the client can understand
- What format the request body is in
- Who the client is
- Which host the client is trying to reach

Common Examples :

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer token
Host: www.coderarmy.in
```

Note :

- Headers are very important in real Spring applications (e.g., for authentication using tokens).
- The server uses headers to understand and process the request correctly.

8 HTTP Response

After processing the request, the server sends back an HTTP response. An HTTP response also has a defined format.

An HTTP response usually contains:

- 1 Status code
- 2 Headers
- 3 Body

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "message": "Login successful",
  "userId": 101
}
```

Common Status Codes:

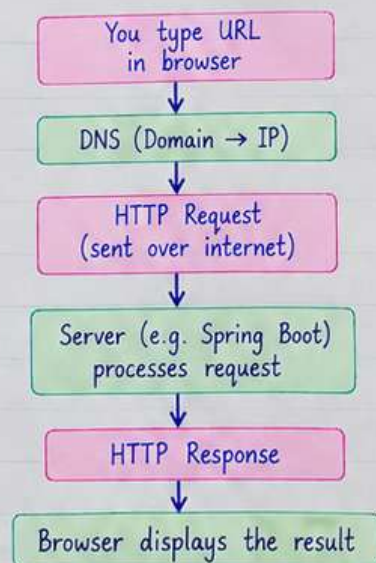
200	→ OK (Success)
201	→ Created
400	→ Bad Request
401	→ Unauthorized
403	→ Forbidden
404	→ Not Found
500	→ Internal Server Error

Status codes help the client understand whether the request was successful or not.

9 What Happens Behind the Scenes?

When you type a URL in the browser, many things happen internally:

- 1 Browser converts the domain name to an IP address (using DNS).
- 2 Browser creates an HTTP request.
- 3 Request is sent over the internet using TCP/IP.
- 4 The server receives the request (e.g., a Spring Boot application).
- 5 The server processes the request (business logic, database, etc.).
- 6 The server sends an HTTP response.
- 7 The browser receives and displays the response.



10 Where Do Spring, Spring Boot and Servlets Fit?

Now that we understand how web applications communicate, let's see where these technologies come into the picture.

• Servlets

- A low-level technology (part of Java).
- Used to handle HTTP requests and responses.
- Provides the basic building blocks for web applications.

• Spring Framework

- A powerful Java framework built on top of Java.
- Provides many modules (e.g., dependency injection, MVC, transaction management, etc.).
- Makes development easier and more structured.

• Spring Boot

- A module of the Spring ecosystem.
- Builds on top of Spring Framework.
- Provides auto-configuration, embedded server, and quick setup.
- Helps us create production-ready applications with minimal configuration.

Spring Boot
(Auto-configuration, easy setup)

Spring Framework
(Core features and modules)

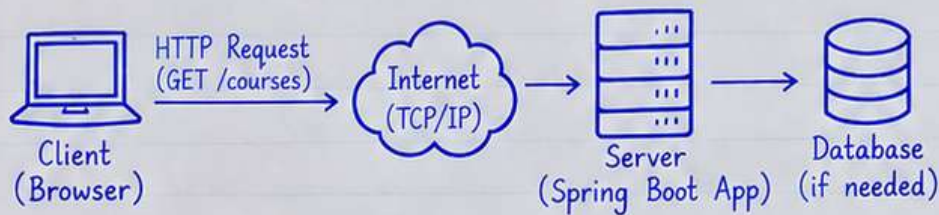
Servlets
(Handle HTTP requests and responses)

Key Takeaway:

- ☆ Spring Boot makes development faster.
- ☆ Spring builds on top of core Java concepts (like Servlets).
- ☆ All of them work together in the Spring ecosystem.

11 How a Request Reaches a Spring Boot Application ?

Let's see the complete flow from browser to Spring Boot application :



Steps :

- 1 Client sends an HTTP request.
- 2 Request travels over the internet (using TCP/IP).
- 3 Server (Spring Boot app) receives the request.
- 4 Spring processes the request (using controllers, services, etc.).
- 5 Server sends an HTTP response back to the client.
- 6 Browser displays or uses the response.

12 Why Do We Need Frameworks Like Spring ?

We could build web applications using only Servlets and plain Java, but it becomes complex and time-consuming.

Challenges without a framework :

- More boilerplate code
- Manual configuration
- Difficult to manage dependencies
- Harder to maintain and scale
- No built-in solutions for common problems (like security, data access, etc.)

How Spring helps ?

- Provides ready-made solutions for common problems.
- Manages configuration and dependency injection.
- Supports integration with databases, security, testing, etc.
- Makes development faster, easier, and more maintainable.

13 Real-Life Analogy

Think of it like a Restaurant :

Without Spring (Using only Servlets)

- You have to buy raw materials.
- You have to cook everything yourself.
- You have to manage staff, cleaning, billing, etc.
- It works, but it's a lot of effort.

With Spring / Spring Boot

- You get a well-equipped kitchen.
- Many things are already set up.
- You can focus on cooking (business logic).
- It saves time and effort.

"Spring doesn't remove the basics, it just makes your life easier by handling the complexity."

14 Summary & Key Takeaways

- 1 A client (browser, app, etc.) sends a request to a server.
- 2 HTTP is the common language used for communication.
- 3 Every interaction follows a request-response cycle.
- 4 An HTTP request contains method, URL, headers and optional body:
- 5 Common HTTP methods: GET, POST, PUT, PATCH, DELETE.
- 6 HTTP headers provide important information about the request.
- 7 Servlets handle HTTP requests and responses at a low level.
- 8 Spring Framework provides core features and modules, and Spring Boot makes it easier to use.

Remember :

- ★ Client asks.
- ★ Server responds.
- ★ HTTP is the language.
- ★ Servlets are the foundation.
- ★ Spring Boot makes development faster.